

Lecture 38: Vision-Language-Action Models

Robots that read

Every number in this deck was checked on 8 August 2026. This field turns over every few months, so a model list older than that is already wrong.

You taught it to see. Now make it do something.

In L33 a model looked at a kitchen photo and described it. In L34 it drew one. Both outputs were the same kind of thing: **tokens a human reads**.

Point the same camera at the same kitchen and say "*put the bowl in the sink*". Now the output has to be a number that a motor can execute, right now, and if it is wrong the bowl breaks.

A **Vision-Language-Action model (VLA)** is a vision-language model fine-tuned so that its output is not text but **robot actions**.

That one change breaks almost everything you have relied on: no ground truth to condition on, no internet-scale corpus, and no way to run the test set twice.

This lecture is deliberately two-sided. The first half is why this is exciting. The second half is what the measurements actually say.

Outline

What a VLA actually is

The shape of the field

There is no internet of robot data

The language channel is largely dead

Embodied red teaming

Can we measure any of this?

What a VLA actually is

One network replaces an entire hand-built robotics stack. That is the whole claim.

Five robotics words, then we can talk normally

You have no robotics background and you do not need one. You need these five.

End effector	The business end of the arm: a gripper, a suction cup, a hand. Policies usually output where the <i>end effector</i> should go, not each joint angle, because that transfers between arms of different shape.
Degrees of freedom	How many numbers you independently control. Arm plus gripper is 7 : three positions, three rotations, one open/close. A humanoid with two five-fingered hands is 40+.
Proprioception	The robot's sense of its own body: current joint angles, gripper width. Cheap, exact, fed in alongside the images.
Teleoperation	A human driving the robot live, through a controller or a small leader arm the real arm mirrors. This is how essentially all training data is made.
Embodiment	A specific robot body. Different cameras, different action dimensions, different reach. "Cross-embodiment" means one model driving several.

The definition, and what it deletes

A VLA is built on **backbones pretrained on large-scale vision-language data**, then fine-tuned to emit control commands. The pretraining is load-bearing: a transformer trained from scratch on robot demonstrations is a visuomotor policy, not a VLA.

The classical stack, four modules, hand-designed interfaces:

perception → object poses
symbolic planner → subgoals
motion planner → trajectory
controller → torques

Every stage has its own representation and its own failure mode.

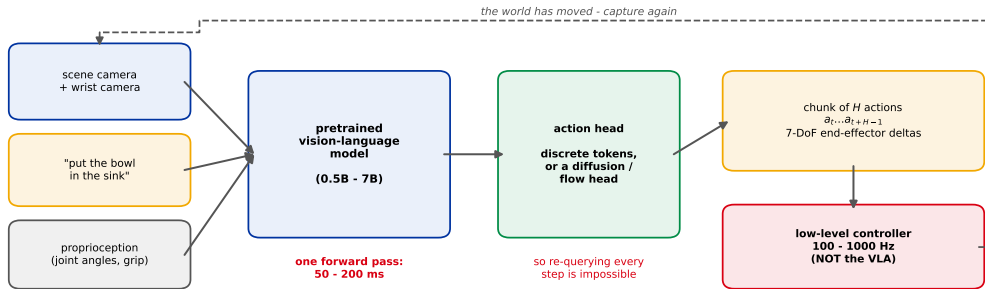
A VLA deletes all four. One network trained end to end on (observation, instruction, action) triples.

Everything else in this lecture, good and bad, is a consequence of that one decision.

The bet: internet-scale visual and linguistic knowledge is what robots were always missing, and once a model has it, *one* set of weights can drive many robots at many tasks.

The loop, once around

From two camera frames and a sentence to a joint command



Two things a language modeller would not expect. The actions are usually **deltas** ("move 2 cm left"), not absolute positions, so the same action means the same thing anywhere in the workspace. And the VLA never closes the torque loop: a conventional controller underneath still does that at hundreds of Hz.

There is no teacher forcing on a robot

In language modelling, a mistake at step t does not corrupt step $t + 1$ during training: you feed the true token. On a robot, an error at step t moves the arm somewhere the demonstration never visited, and step $t + 1$ is now out of distribution.

Compounding error. Behaviour cloning drifts, and the drift feeds itself. This has no analogue in ordinary supervised learning, where a wrong prediction on row 7 does not change row 8.

Action chunking is the fix, from ACT (Zhao et al., 2023). Predict the next H actions in one shot and execute them open-loop before asking again. Typical H is 8 to 50; pi-0 uses $H = 50$.

Error now accumulates over T/H decisions instead of T . It also lets the model commit to one intent instead of averaging over a hesitating human's several.

The cost is reactivity. During a chunk the robot is blind. And there is a visible jerk every time a new chunk starts.

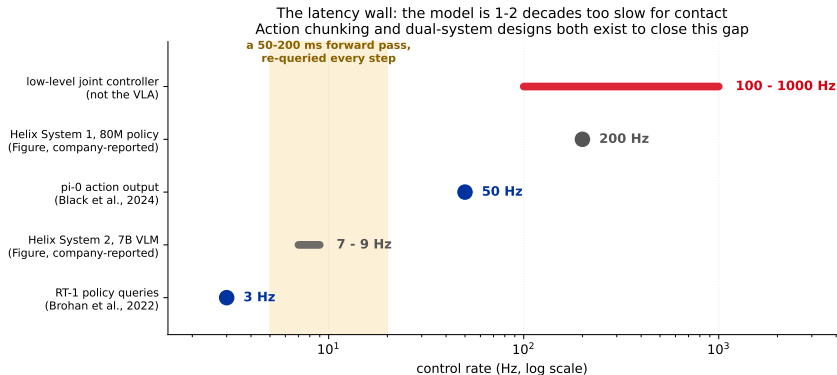
Three ways to make a language model emit a number

Discretize, or model the continuous distribution directly? The same question as ch10.

	How	Why it wins or loses
Discrete bins RT-2, OpenVLA	Cut each action dimension into 256 bins, treat each bin as a token, autoregress. OpenVLA overwrites the 256 least-used Llama-2 tokens.	Zero architectural change: the action head <i>is</i> the language head. But 7 dimensions $\times$ 50 steps is 350 tokens, far too slow.
Compressed tokens FAST	Discrete cosine transform along the chunk, drop the small high-frequency terms, then byte-pair encode.	Robot trajectories are smooth signals, so treat them like audio. Roughly 10x compression, 5x faster training.
Continuous head Octo, pi-0, GR00T N1	A separate head trained as conditional diffusion or flow matching, conditioned on the backbone's features.	Demonstrations are genuinely multimodal: two humans reach around a mug two ways, and averaging goes <i>through</i> the mug.

FAST is Frequency-space Action Sequence Tokenization (Pertsch et al., 2025). The trick is borrowed straight from JPEG.

Too slow to touch anything



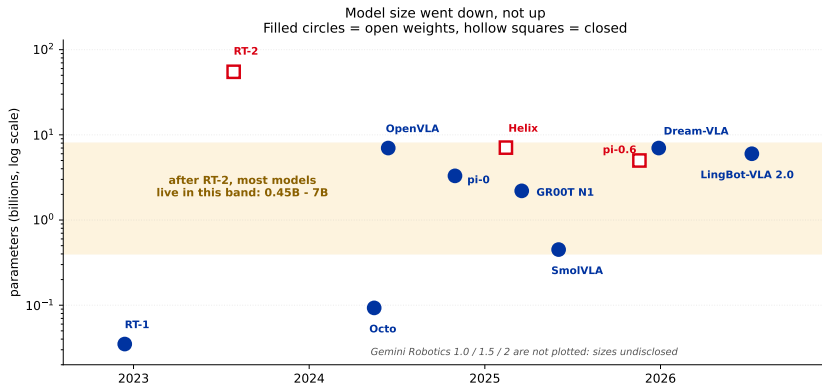
Dual-system designs. A slow semantic model and a fast reactive one. Figure's Helix runs a 7B model at 7-9 Hz that compresses the scene into a *single latent vector*, driving an 80M policy at 200 Hz. GR00T N1 has the same shape.

Real-time chunking (Black et al., 2025). Compute the next chunk while executing the current one, freeze the actions already committed, and *inpaint* the rest. Yes, that is ch10's inpainting, applied to a trajectory.

The shape of the field

Five years of models, and a size chart that looks nothing like the language one.

The chart that does not look like the language one

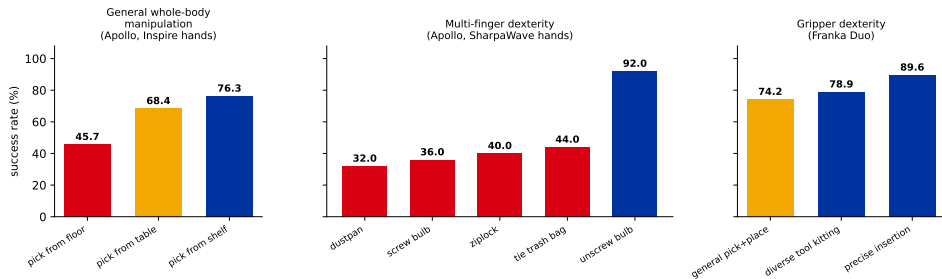


Two hard caps. The model has to run *on the robot* at control rate, which bounds it at a few billion parameters. And with about a million trajectories in the world, a 7B model already overfits comfortably.

RT-2 (2023) was 55B and ran at 3 Hz, so it was never deployable. OpenVLA (Kim et al., 2024) beat it by **16.5 points absolute across 29 tasks** at 7B, and became the field's default baseline.

Demo, company-reported, or measured benchmark?

The frontier, in the vendor's own numbers - Gemini Robotics 2, 30 July 2026



Company-reported, self-chosen tasks, protocol and trial counts not published. Screwing a bulb in: 36%. Unscrewing one: 92%.

Robotics publishes an unusual amount of marketing, so separate three things every time. A **demo** is a video with no number (a robot folding laundry for eighteen hours tells you the system does not fall over, not its success rate). A **company-reported** number is real but unreproducible: self-chosen tasks, unpublished protocol. A **measured benchmark** is something a second lab can run. The chart above is the middle kind, and it is more honest than most: screwing a bulb *in* is 36%.

"Generalist policy" is three claims, not one

They travel under one label and they are not equally supported. Keep them apart.

1. Multi-task.

One set of weights, many tasks in one environment, selected by the instruction.

Solved since RT-1 in 2022: 700+ instructions, one model.

2. Cross-embodiment.

One set of weights drives *different robots*.

Mostly works. Open X-Embodiment pooled 60 datasets and got positive transfer. Conditions apply - see later.

3. Open-world.

New objects, new scenes, new instructions, zero-shot.

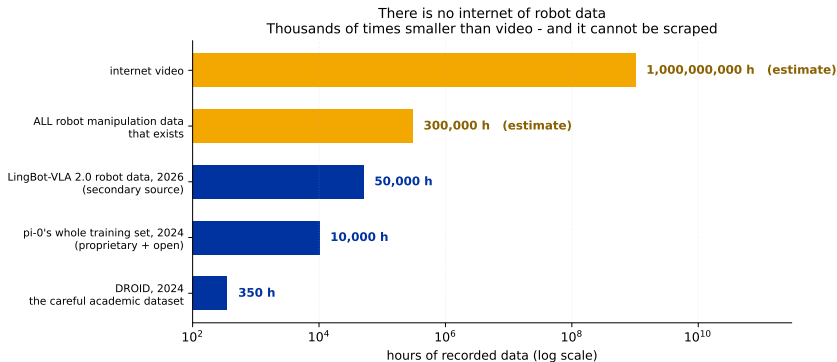
The research frontier, and where the marketing lives. This is the claim that justifies the whole enterprise.

The 2026 review of 164 VLA submissions to one conference is blunt: open models match the frontier *in simulation* and "dramatically lag" it in zero-shot open-world behaviour.

There is no internet of robot data

In vision and language, architecture was the bottleneck and data was free.
Here it is exactly reversed.

The gap



Orange bars are industry estimates (Bessemer; the gogoduck912 robotics scaling-law analysis), not measurements.

And it cannot be closed by scraping, because the data does not exist to be scraped. It has to be generated by a human moving a robot in real time: roughly one human-hour per robot-hour. That is a structural fact, not an engineering backlog. Every strange decision in this field is a response to it.

Four ways to manufacture what you cannot scrape

Teleoperation	The gold standard and the expensive one. An ALOHA bimanual rig is around \$20k; GELLO does the same job with a 3D-printed exoskeleton for about \$300. DROID took 50 collectors at 13 institutions 12 months to reach 350 hours.
Human video	The largest available signal, with no action labels and a hand that is not a gripper. The 2026 answer is retargeting wrist and hand poses into pseudo-actions. EgoScale pretrained on 20,854 hours of egocentric video and reports a <i>log-linear scaling law</i> between human-action loss and data scale. One paper, one lab, but it is the first credible route to a corpus that scales.
Simulation	Nearly free per sample, with resets and ground truth. Rendering is close to solved; friction, deformation and contact are not, and contact is the hard part.
Data engines	Systems that <i>manufacture</i> demonstrations. MimicGen turns 200 human demonstrations into 50,000 by replaying them onto new object poses. Huge multiplier, but the diversity is combinatorial, not semantic: same skill, new places, no new skills.

Sim-to-real: what actually transfers

Folklore until recently. Jin et al. (2026) ran **over 10,000 real-world trials** to ablate it, which is roughly 400 times more than a typical paper affords.

- ▶ **Spatial randomization beats appearance randomization.** Randomizing camera pose and table height helped more than randomizing backgrounds and lighting.
- ▶ Randomizing **per frame** beats per episode.
- ▶ Rendering fidelity helps, then plateaus.
- ▶ **Reinforcement learning (ch11) is the big lever:** 5.6% → 33.4% real-world success, and 42.8% combined with randomization.

Counter-intuitive, and useful: everyone reaches for *visual* domain randomization first because it is easy, and it is the less valuable axis.

Read 42.8% again. That is the *good* number, after the best recipe in the largest study of its kind. Sim-to-real for manipulation remains bad in absolute terms.

Cross-embodiment transfer is conditional, not general

Pooling robots usually helps. Open X-Embodiment (2023) merged 60 datasets from 34 labs into 1M+ trajectories across 22 embodiments, and the models trained on it beat their own single-robot baselines. Gemini Robotics 1.5's Motion Transfer is a whole mechanism built to move a skill from one body to another.

But negative transfer is reported. Embodiments with few close relatives in the pool can do *worse* than training alone, when a lot of low-quality data meets high body diversity.

Reported in the 2026 cross-embodiment literature; I could not trace the specific robots to a paper I read, so treat the direction as the claim, not a number.

The honest framing: transfer is a similarity-weighted average, not a free lunch. Pooling helps bodies with close relatives in the pool and can hurt the ones without.

You have seen this before: multi-task learning helps until task conflict dominates.

So "one brain, any robot" is really "one brain, any robot sufficiently similar to something in the training set". That is a much weaker claim, and it changes the economics.

The language channel is largely dead

This is the sequel to L33's hallucination frame, and it is worse than it was there.

Predict first

A team reports that their VLA's success rate is **almost unchanged** when you rephrase the instruction. "Open the drawer", "pull the drawer open", "could you open that drawer" all score the same.

Good news or bad news?

Predict first

A team reports that their VLA's success rate is **almost unchanged** when you rephrase the instruction. "Open the drawer", "pull the drawer open", "could you open that drawer" all score the same.

Good news or bad news?

It reads like robustness. The model understood the request rather than memorising a string. This is how such a result is usually reported.

It is evidence the channel is dead. A model that is insensitive to *every* change in wording may not be reading the wording at all.

The test that separates the two: change the wording so the *meaning* changes, and check that the behaviour changes too.

The experiment that settles it

LIBERO-PRO (Zhou et al., 2026) takes the standard benchmark and perturbs four things: the object, its starting position, the instruction wording, and the environment.

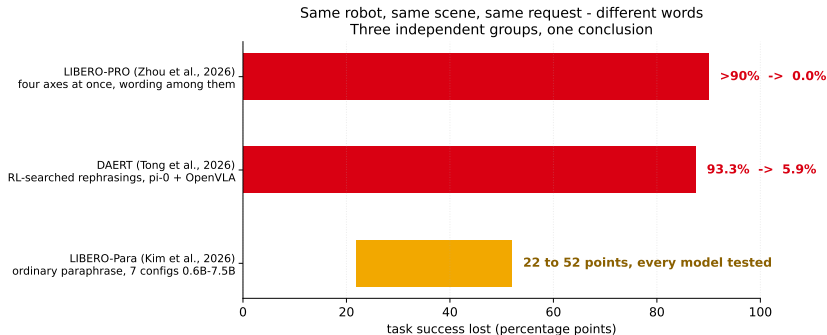
Models that score **over 90%** on standard LIBERO score **0.0%** under the generalized setting. Not degraded. Zero.

The diagnostic is better than the headline. Replace the instruction with a **meaningless string** - the paper's examples are `fdsgfdsgsd` and `xxx` - and the model still executes nearly the same trajectory.

The policy is not reading the instruction. It is matching the *scene* to a memorised trajectory. The authors' own word for the reported accuracies is "rote recall".

They also find a hard positional cliff: success collapses once the object is displaced past a small threshold, and stays at zero.

It is not one benchmark having a bad day



Different papers, different models, different perturbation strengths. The shared quantity is points of success lost, not a ranking.

DAERT is Diversity-Aware Embodied Red Teaming (Tong et al., 2026): train an attacker with reinforcement learning to *search* for rephrasings that break the policy. No adversarial patch, no jailbreak - just different English words for the same request.

Human raters judge the generated instructions **natural**. A related 2026 method also shows a partial defence: fine-tuning on generated adversarial instructions recovers some ground on *unseen* ones.

L33 said the prior outvotes the image. Here it is worse.

In L33, a vision-language model hallucinated a refrigerator into a kitchen photo: the **language prior** was trained on trillions of tokens and easily outvoted 576 blurry visual vectors.

Flip it around. In a VLA the **vision** pathway carries the policy, and the language input can be ignored **wherever the training data never forced it to matter**. Swap the object named in the instruction and the model executes the original task anyway.

Language still *selects* among the task templates it was trained on - that is the 700-instruction result from three slides ago, and it is real. What it cannot do is **override what the scene suggests**, which is exactly what a new instruction is for.

Two structural causes, both fixable in principle:

1. The training instructions are templated. If every demonstration says "pick up the X and place it in the Y", the model learns the slot filler, not the sentence. Everyone optimised task diversity; nobody optimised linguistic diversity.

2. Vision wins every conflict. Build a scene where the instruction contradicts the visual prior (Fang et al., 2026) and models systematically follow the picture, even when following the text was correct.

A 2026 ablation study points the same way from the architecture side: the *visual* module is the bottleneck, and a better general vision-language model does not reliably make a better controller.

In Armenian it would be much worse

Dong et al. (2026) translated the standard benchmark into ten languages and measured **success-rate drops of 30 to 50% on non-English instructions.**

Their more interesting finding is **step-wise** sensitivity: language dependence is wildly non-uniform along a trajectory. A few steps dominate all the failure; most steps are effectively language-agnostic. Their fix intervenes only at those steps.

A concrete prediction you can check. Hand an open VLA an instruction in Armenian and it will very probably fail - and now you can say roughly why, and roughly *where* in the trajectory.

What would count as fixed. A model that is flat under adversarial paraphrase search *and* whose success collapses appropriately when the instruction is replaced with nonsense. Both directions together. Nobody has shown this.

Embodied red teaming

A new category of attack: nobody jailbreaks anything. The attacker moves an object.

Token space is not configuration space

Everything you know about model safety was learned on text, and it does not carry over.

A model that refuses to *describe* how to knock a glass off a table will happily emit the action trajectory that knocks it off.

The safety constraint was learned in **token space**. The harm happens in **configuration space**.

That framing is from SafeVLA (Zhang et al., 2025), which formulates safety as a constrained Markov decision process and optimises against elicited unsafe behaviours.

The 2026 framing is sharper still: risk "has fundamentally shifted from the intent space to the physical space". The intent stays benign throughout.

RedVLA: the attacker moves a knife

RedVLA (Zhang et al., April 2026) is a red-teaming framework for *physical* safety. Two stages, and neither touches the prompt.

1. Risk scenario synthesis. Find the critical interaction regions along a perfectly benign trajectory. Inject a risk object - a knife, a bottle - into one of them, keeping the task feasible and the instruction unchanged.

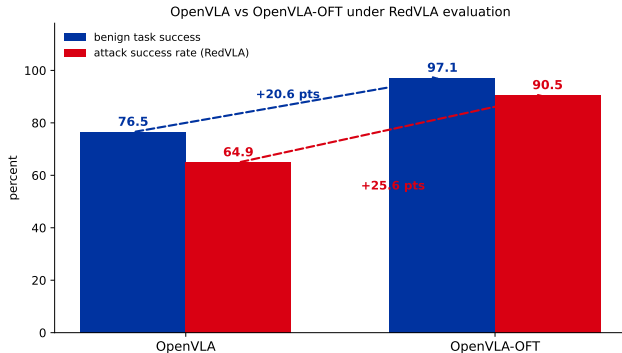
2. Trajectory-driven amplification. Gradient-free search over *where to put it*, refined until it reliably triggers unsafe behaviour. Gradient-free, so it works across model families.

Attack success rate, six models, LIBERO:
64.9% (OpenVLA) to **95.5%** (pi-0.5), within 10 optimisation iterations.

Validated sim-to-real on a physical Franka arm as well.

The scene looks benign. The instruction *is* benign. Nothing in the input would trip a text filter.

The better policy is the more attackable one



The harm happens while the robot is working. Across all six models most unsafe rollouts are "Success + Unsafe" or "Attempt + Unsafe" - the policy completed or was actively pursuing the task *and* caused harm on the way. Outright collapse is only a small fraction.

So task success rate is not a safety metric, and here it is anti-correlated with one. Every leaderboard in this lecture would score those rollouts as wins.

Defences cost capability, and the robots are already shipping

What exists. Runtime monitors that halt execution before an unsafe action; constrained-optimisation safety training; external safety controllers.

RedVLA's own monitor cuts attack success by roughly two thirds and costs about 4 points of benign performance. SafeVLA reports the friendlier result - an 83.6% cut in cumulative violation cost with task success *up* 3.85% - but on its own procedurally generated benchmark.

What does not exist. Formal verification of a neural policy. Certification. Any accepted standard for what a VLA must be tested against before it works near a person.

The 2026 consensus is *layered* mitigation, which is an admission that the policy itself cannot be trusted.

Put these two facts on the same slide, because they are true at the same time. Humanoids are in early commercial deployment and the research literature reports attack success rates above 90% on the same class of model. An industrial cell with cages and interlocks is a different risk category from a kitchen, and the field is moving toward the kitchen.

Can we measure any of this?

Including the evidence in the last two sections. Especially that.

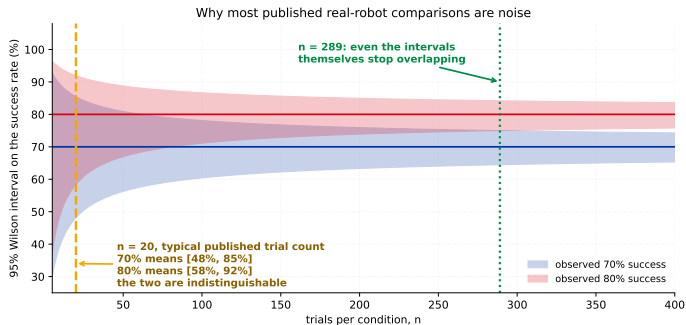
The test set is a room

Simulation is a proxy, and the saturated kind: LIBERO now sits at 95-98% and the difference between 98% and 99% carries no information. So the claims that matter are made on physical robots, where evaluation has problems with no analogue anywhere else in machine learning.

- ▶ **The test set is a physical room.** Lighting drifts through the day, objects get placed slightly differently, the gripper wears. Another lab cannot reproduce your evaluation because they do not have your room.
- ▶ **Every trial costs a minute and a human reset.** So n is tiny.
- ▶ **There is no held-out set** in any meaningful sense.

PhAIL (Arkhangelskiy, 2026) states the practice plainly: real-world VLA evaluation "still rests on binary success rate at a fixed timeout with $N \leq 25$ rollouts per condition, almost always **without confidence intervals or paired statistical comparison**".

Do the arithmetic



Exact 95% Wilson intervals. A pooled two-proportion z-test is less conservative and separates 70% from 80% at $n = 145$ per arm - still 7x the $n = 20$ annotated above.

Two policies reported at **70%** and **80%** on 20 trials are statistically indistinguishable. A very large share of published VLA comparisons sits inside that noise band - which means the ranking, and sometimes the paper's conclusion, could flip on a rerun.

Now turn it on this lecture

Everything in the last two sections is subject to its own critique, and a course that taught you to check numbers should say so out loud.

The bad news is measured on the same weak instruments. The 0.0%, the 93.3% → 5.9%, the attack success rates - almost all of it is *simulation*, on the benchmark the same authors just told you is saturated.

Why it still stands. The direction is unanimous across at least five independent groups with different methods, and the gibberish-instruction result needs no statistics at all: either the trajectory changes or it does not.

How to read any VLA result, in six questions:

1. Simulation or real? If real, what is n ?
2. Which LIBERO? 97% on the plain one means almost nothing in 2026.
3. Was the instruction varied at all?
4. Was the camera pose or object placement varied?
5. Company blog or paper?
6. Is a demo being used as evidence?

Recap

1. A VLA is a vision-language model whose output is a motor command. It deletes the four-stage classical stack; everything else follows from that.
2. Action chunking exists because there is no teacher forcing on a robot. Dual-system designs exist because a 3B forward pass is a decade too slow for contact.
3. Data is the whole problem. The corpus is three orders of magnitude short of video and cannot be scraped.
4. **The language channel is largely dead.** 90%+ collapses to 0.0%, and gibberish instructions produce the same trajectory.
5. **Embodied red teaming is a new risk category.** Capability and vulnerability rise together, and the robot succeeds *dangerously*.
6. **And we cannot measure any of it well.** $N \leq 25$ trials, no intervals - which cuts against finding 4's own benchmarks too.

Next: the chapter page carries the reading list and the sources behind every number in this deck.

Three observations would make this deck out of date, and all three are live: a data source that scales without human robot-time; a model flat under paraphrase search *and* broken by nonsense; one shared real-robot evaluation a second lab can reproduce inside stated intervals.

And yet the trajectory is steep. RT-1 was under four years ago, ran at 3 Hz, on one robot, in one kitchen.